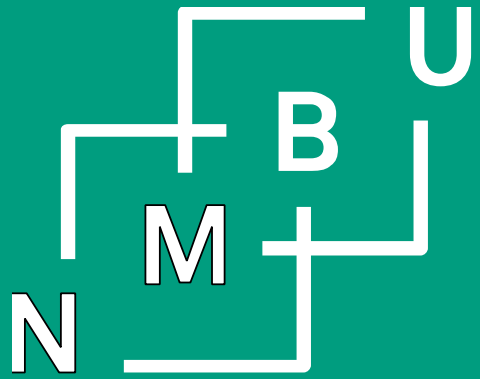
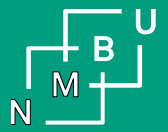



Norwegian University
of Life Sciences




# Pipelines

Chapter 6 in the Python Machine Learning book

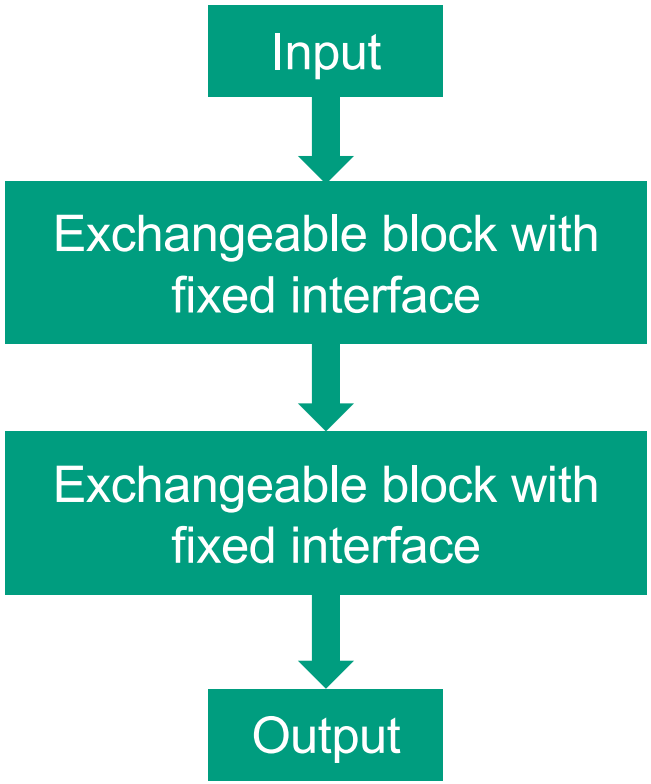
# What is a pipeline?

- Any suggestions?
- Pipelines are tools to streamline and automate machine learning workflows
- They enable chaining of data preprocessing and modelling steps
- Ensure consistency and reproducibility of machine learning models

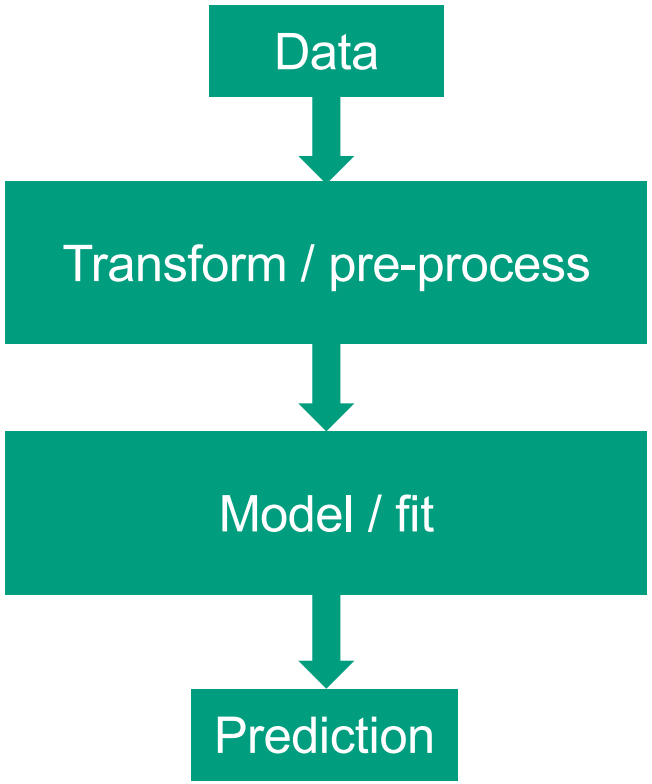
Pipelines in scikit-learn:
http://scikit-learn.org/stable/modules/pipeline.html

# Pipelines



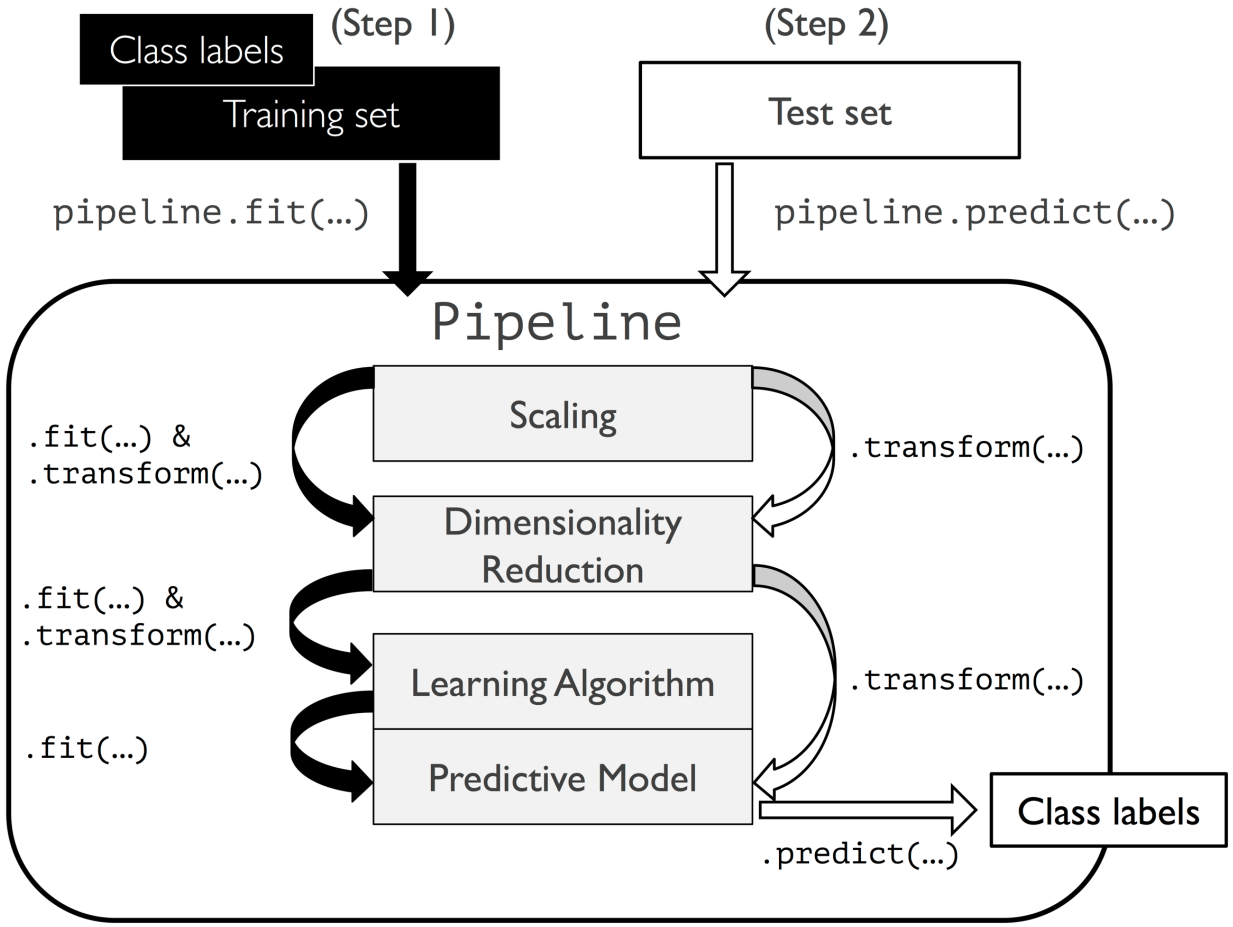



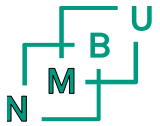

General Pipeline Concept




Machine Learning Pipeline

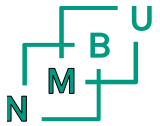
# Pipelines


Class labels
(Step 1)
Training set
(Step 2)
Test set
pipeline.fit(...)
pipeline.predict(...)
Pipeline
Scaling
.fit(...) &
.transform(...)
.transform(...)
Dimensionality
Reduction
.fit(...) &
.transform(...)
Learning Algorithm
.transform(...)
.fit(...)
Predictive Model
Class labels
.predict(...)




# Why use pipelines?

- Simplifies workflow management
- Prevents data leakage by ensuring preprocessing steps are applied to training data separately from testing data
- Enhances code readability and maintainability
- Facilitates hyperparameter tuning




# Advantages of pipelines

- **Sequencing**: Pipelines allow for defining a sequence of data processing steps and ML algorithms
- **Consistency**: Pipelines ensure that all preprocessing steps are applied consistently and ensures reproducibility
- **Automation:** Automatically applies transformations to test data
- **Convenience**: Pipelines provide a convenient way to fit, predict, and evaluate models with a single call
- **Modularization:** Easily swap or add components
- **Hyperparameter tuning**: They are especially convenient when tuning hyperparameters. Efficient search over a wide range of hyperparameters.

# Using a pipeline

- Fitting the pipeline:

```
pipeline.fit(X_train, y_train)
```

- Making predictions:

```
y_pred = pipeline.predict(X_test)
```

- All steps are executed sequentially.

# Pipelines for hyperparameter tuning

- Example usage:

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'classifier__C': [0.1, 1, 10]
}

grid = GridSearchCV(pipeline, param_grid, cv=5)
grid.fit(X_train, y_train)
```

# Pipelining multiple preprocessing steps

- Pipelines can include multiple preprocessing steps:

```
from sklearn.impute import SimpleImputer

pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler()),
    ('classifier', LogisticRegression())
])
```

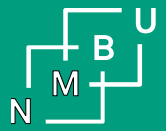
# Feature union with pipelines

- Combining features from different preprocessing steps:

```
from sklearn.compose import ColumnTransformer

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), numerical_features),
    ('cat', OneHotEncoder(), categorical_features)
])
```

- Integrated into a pipeline.


U
B
M
N